

1. ALSU Design

Shift Register instantiation:

```
1
2 //Output of shift register
3 wire [5:0] out_shift_reg;
4
5 shift_reg shift_reg_dut(.serial_in(serial_in_reg), .direction(direction_reg), .mode(opcode_reg[0]), .datain(out), .dataout(out_shift_reg));
6
```

Shift Register output assignment to ALSU Assignment:

```
1 3'h2: out <= (FULL_ADDER == "ON")? A_reg + B_reg + cin_reg : A_reg + B_reg ; //Fixed Bug: added FULL_ADDER checker
2 3'h3: out <= A_reg * B_reg;
3 3'h4: begin
4     // if (direction_reg)
5     //   out <= {out[4:0], serial_in_reg};
6     // else
7     //   out <= {serial_in_reg, out[5:1]};
8     out <= out_shift_reg;
9 end
10 3'h5: begin
11     // if (direction_reg)
12     //   out <= {out[4:0], out[5]};
13     // else
14     //   out <= {out[0], out[5:1]};
15     out <= out_shift_reg;
```

2. Shift Register Design

```
1 module shift_reg (serial_in, direction, mode, datain, dataout);
2 input serial_in, direction, mode;
3 input [5:0] datain;
4 output reg [5:0] dataout;
5
6 always @(*) begin
7     // if (reset)
8     //   dataout = 0;
9     // else
10     if (mode) // rotate
11         if (direction) // left
12             dataout = {datain[4:0], datain[5]};
13         else
14             dataout = {datain[0], datain[5:1]};
15     else // shift
16         if (direction) // left
17             dataout = {datain[4:0], serial_in};
18         else
19             dataout = {serial_in, datain[5:1]};
20 end
21 endmodule
```

3. Shift Register Interface

```
1 interface shift_reg_if ();
2     logic serial_in, direction, mode;
3     logic [5:0] datain, dataout;
4 endinterface : shift_reg_if
```

4. Top

```
1 `include "ALSU/uvm/includes/all_imports.svh"
2
3 module top();
4
5     bit clk;
6
7     initial begin
8         clk = 0;
9         forever
10             #1 clk = ~clk;
11     end
12
13     alsu_if alsuif (clk);
14     shift_reg_if shiftif ();
15
16     ALSU DUT (.clk(clk), .A(alsuif.A), .B(alsuif.B),
17             .cin(alsuif.cin), .serial_in(alsuif.serial_in),
18             .red_op_A(alsuif.red_op_A), .red_op_B(alsuif.red_op_B),
19             .opcode(alsuif.opcode), .bypass_A(alsuif.bypass_A),
20             .bypass_B(alsuif.bypass_B), .rst(alsuif.rst), .direction(alsuif.direction),
21             .leds(alsuif.leds), .out(alsuif.out));
22
23     //bind the SVA module to the designs
24     bind ALSU ALSU_SVA ALSU_SVA_inst(.clk(clk), .A(alsuif.A), .B(alsuif.B),
25             .cin(alsuif.cin), .serial_in(alsuif.serial_in),
26             .red_op_A(alsuif.red_op_A), .red_op_B(alsuif.red_op_B),
27             .opcode(alsuif.opcode), .bypass_A(alsuif.bypass_A),
28             .bypass_B(alsuif.bypass_B), .rst(alsuif.rst), .direction(alsuif.direction),
29             .leds(alsuif.leds), .out(alsuif.out));
30
31
32     assign shiftif.serial_in = DUT.serial_in_reg;
33     assign shiftif.direction = DUT.direction_reg;
34     assign shiftif.mode = DUT.opcode[0];
35     assign shiftif.datain = DUT.out;
36
37     initial begin
38         uvm_config_db#(virtual alsu_if)::set(null, "uvm_test_top", "alsu_vif", alsuif);
39         uvm_config_db#(virtual shift_reg_if)::set(null, "uvm_test_top", "shift_vif", shiftif);
40         run_test("alsu_test");
41     end
42
43 endmodule
```

5. Test

```
1 package alsu_test_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5
6 import alsu_env_pkg::*;
7 import alsu_config_pkg::*;
8 import alsu_seq_pkg::*;
9
10 import shift_reg_env_pkg::*;
11 import shift_reg_config_pkg::*;
12
13
14 class alsu_test extends uvm_test;
15     `uvm_component_utils(alsu_test)
16
17     //ALSU
18     alsu_env my_alsu_env;
19     alsu_config alsu_config_obj_test;
20
21     //Shift reg
22     shift_reg_env my_shift_reg_env;
23     shift_reg_config shift_reg_config_obj_test;
24
25     //
26     alsu_reset_seq reset_seq;
27     alsu_main_seq main_seq;
28     alsu_directed_seq direct_seq;
29
30     function new(string name = "alsu_test", uvm_component parent = null);
31         super.new(name, parent);
32     endfunction
33
34     function void build_phase(uvm_phase phase);
35         super.build_phase(phase);
36         my_alsu_env = alsu_env::type_id::create("my_alsu_env", this);
37         my_shift_reg_env = shift_reg_env::type_id::create("my_shift_reg_env", this);
38         reset_seq = alsu_reset_seq::type_id::create("reset_seq");
39         main_seq = alsu_main_seq::type_id::create("main_seq");
40         direct_seq = alsu_directed_seq::type_id::create("direct_seq");
41         alsu_config_obj_test = alsu_config::type_id::create("alsu_config_obj_test");
42         shift_reg_config_obj_test = shift_reg_config::type_id::create("shift_reg_config_obj_test");
43         if(!uvm_config_db #(virtual alsu_if)::get(this, "", "alsu_vif", alsu_config_obj_test.alsu_config_vif))
44             `uvm_fatal("build_phase", "Test - Unable to get Als Virtual Interface");
45
46         if(!uvm_config_db #(virtual shift_reg_if)::get(this, "", "shift_vif", shift_reg_config_obj_test.shift_reg_vif))
47             `uvm_fatal("build_phase", "Test - Unable to get Als Virtual Interface");
48
49         alsu_config_obj_test.is_active = UVM_ACTIVE;
50         shift_reg_config_obj_test.is_active = UVM_PASSIVE;
51
52         uvm_config_db#(alsu_config)::set(null, "*", "ALSU_CFG", alsu_config_obj_test);
53         uvm_config_db#(shift_reg_config)::set(null, "*", "SHIFT_REG_CFG", shift_reg_config_obj_test);
54
55     endfunction
56
57     task run_phase(uvm_phase phase);
58         super.run_phase(phase);
59         phase.raise_objection(this);
60
61         `uvm_info("run_phase", "Reset Asserted", UVM_MEDIUM);
62         reset_seq.start(my_alsu_env.agt.sqr);
63         `uvm_info("run_phase", "Reset Deasserted", UVM_MEDIUM);
64
65         `uvm_info("run_phase", "Main Sequencer Begin", UVM_MEDIUM);
66         main_seq.start(my_alsu_env.agt.sqr);
67         `uvm_info("run_phase", "Main Sequencer end", UVM_MEDIUM);
68
69         `uvm_info("run_phase", "Directed Sequencer Begin", UVM_MEDIUM);
70         direct_seq.start(my_alsu_env.agt.sqr);
71         `uvm_info("run_phase", "Directed Sequencer end", UVM_MEDIUM);
72
73         phase.drop_objection(this);
74     endtask
75 endclass
76
77 endpackage
```

6. Configuration Object

ALSU:

```
1  package alsu_config_pkg;
2      import uvm_pkg::*;
3      `include "uvm_macros.svh"
4
5      class alsu_config extends uvm_object;
6          `uvm_object_utils(alsu_config)
7          // Configuration parameters
8          virtual alsu_if alsu_config_vif;
9          uvm_active_passive_enum is_active;
10
11          function new(string name = "alsu_config");
12              super.new(name);
13          endfunction
14      endclass : alsu_config
15  endpackage
```

Shift Register:

```
1  package shift_reg_config_pkg;
2      import uvm_pkg::*;
3      `include "uvm_macros.svh"
4
5      class shift_reg_config extends uvm_object;
6          `uvm_object_utils(shift_reg_config)
7
8          virtual shift_reg_if shift_reg_vif;
9          uvm_active_passive_enum is_active;
10
11
12          function new(string name = "shift_reg_config");
13              super.new(name);
14          endfunction
15
16
17      endclass
18  endpackage
19
```

7. ALSU agent

```
1  package alsu_agent_pkg;
2
3      import uvm_pkg::*;
4      `include "uvm_macros.svh"
5      import alsu_driver_pkg::*;
6      import alsu_sequencer_pkg::*;
7      import alsu_monitor_pkg::*;
8      import alsu_config_pkg::*;
9      import alsu_seq_item_pkg::*;
10
11
12  class alsu_agent extends uvm_agent;
13      `uvm_component_utils(alsu_agent)
14
15      alsu_config alsu_cfg;
16      alsu_driver drv;
17      alsu_monitor mon;
18      alsu_sequencer sqr;
19      uvm_analysis_port #(alsu_seq_item) agt_ap;
20
21
22  function new(string name = "alsu_agent", uvm_component parent = null);
23      super.new(name, parent);
24  endfunction //new()
25
26  function void build_phase(uvm_phase phase);
27      super.build_phase(phase);
28      if(!uvm_config_db #(alsu_config)::get(this, "", "ALSU_CFG", alsu_cfg))begin
29          `uvm_fatal("build_phase", "Error, Unable to get configuration object")
30      end
31      if(alsu_cfg.is_active == UVM_ACTIVE)begin
32          drv = alsu_driver::type_id::create("drv", this);
33          sqr = alsu_sequencer::type_id::create("sqr", this);
34      end
35      mon = alsu_monitor::type_id::create("mon", this);
36      agt_ap = new("agt_ap", this);
37  endfunction
38
39  function void connect_phase(uvm_phase phase);
40      super.connect_phase(phase);
41      //driver connections
42      if(alsu_cfg.is_active == UVM_ACTIVE)begin
43          drv.alsu_vif = alsu_cfg.alsu_config_vif;
44          drv.seq_item_port.connect(sqr.seq_item_export);
45      end
46      //monitor connections
47      mon.alsu_vif = alsu_cfg.alsu_config_vif;
48      mon.mon_ap.connect(agt_ap);
49  endfunction
50  endclass //alsu_agent extends uvm_agent
51
52  endpackage
```

8. Shift Register Agent

```
1  package shift_reg_agent_pkg;
2
3      import uvm_pkg::*;
4      import shift_reg_config_pkg::*;
5      import shift_reg_seq_item_pkg::*;
6      import shift_reg_driver_pkg::*;
7      import shift_reg_monitor_pkg::*;
8      import shift_reg_sequencer_pkg::*;
9      `include "uvm_macros.svh"
10
11  class shift_reg_agent extends uvm_agent;
12      `uvm_component_utils(shift_reg_agent)
13      shift_reg_sequencer sequencer;
14      shift_reg_driver driver;
15      shift_reg_monitor monitor;
16      shift_reg_config shift_reg_cfg;
17      uvm_analysis_port #(shift_reg_seq_item) agt_ap;
18
19
20      function new(string name = "shift_reg_agent", uvm_component parent = null);
21          super.new(name, parent);
22      endfunction //new()
23
24      function void build_phase(uvm_phase phase);
25          super.build_phase(phase);
26
27          if(!uvm_config_db #(shift_reg_config)::get(this, "", "SHIFT_REG_CFG", shift_reg_cfg))
28              `uvm_fatal("build_phase", "Unable to get configuration object");
29          if(shift_reg_cfg.is_active == UVM_ACTIVE)begin
30              driver = shift_reg_driver::type_id::create("driver", this);
31              sequencer = shift_reg_sequencer::type_id::create("sequencer", this);
32          end
33          monitor = shift_reg_monitor::type_id::create("monitor", this);
34          agt_ap = new("agt_ap",this);
35      endfunction
36
37      function void connect_phase(uvm_phase phase);
38          super.connect_phase(phase);
39          if(shift_reg_cfg.is_active == UVM_ACTIVE)begin
40              driver.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
41              driver.seq_item_port.connect(sequencer.seq_item_export);
42          end
43          monitor.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
44          monitor.mon_ap.connect(agt_ap);
45      endfunction
46
47      endclass //shift_reg_agent extends uvm_agent
48
49  endpackage
```

9. Assertions

```
1 import alsu_seq_item_pkg::*;
2 module ALSU_SVA(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
3
4 input logic clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5 input logic [2:0] opcode;
6 input logic signed [2:0] A, B;
7 input logic [15:0] leds;
8 input logic signed [5:0] out;
9
10 logic invalid_opcode, invalid_red_op, invalid;
11
12 assign invalid_red_op = (red_op_A | red_op_B) & (opcode[1] | opcode[2]);
13 assign invalid_opcode = opcode[1] & opcode[2];
14 assign invalid = invalid_red_op | invalid_opcode;
15
16 // Reset behavior
17 always_comb begin
18     //5.When the asynchronous reset is asserted then the counter output is tied to low at the same instant.
19     if(rst) begin
20         assert final (out == 0);
21         cover final (leds == 0);
22     end
23
24 end
25
26
27 // Invalid case: out must be zero
28 property p_invalid_out_zero;
29     @(posedge clk) disable iff (rst) (invalid && !bypass_A && !bypass_B) |-> ##2 (out == 0);
30 endproperty
31
32 property p_valid_leds_zero;
33     @(posedge clk) disable iff (rst) (!invalid) |> (leds == 0);
34 endproperty
35
36
37 property bypassA_prop;
38     @(posedge clk) disable iff (rst) bypass_A |>##1 (out == $past(A,2));
39 endproperty
40
41 property bypassB_prop;
42     @(posedge clk) disable iff (rst) bypass_B && !bypass_A |>##1 (out == $past(B,2));
43 endproperty
44
```

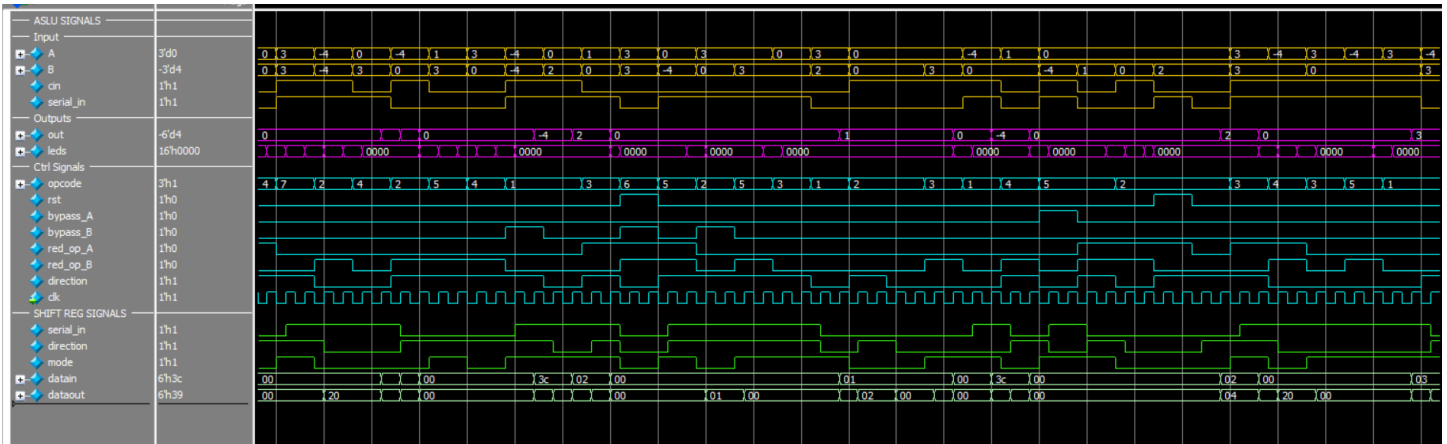
```

1
2
3     sequence NO_bypass;
4     !(bypass_A || bypass_B);
5 endsequence
6
7 // Red_op_A assertion
8 property red_op_A_xor;
9 @(posedge clk) disable iff(rst)
10     NO_bypass |->(red_op_A && opcode==XOR) | => ##1 (out == $past(^A,2));
11 endproperty
12
13 property red_op_A_or;
14 @(posedge clk) disable iff(rst)
15     NO_bypass |->(red_op_A && opcode==OR) | => ##1 (out == $past(|A,2));
16 endproperty
17
18
19 // Red_op_B assertion - XOR
20 property red_op_B_xor;
21 @(posedge clk) disable iff(rst)
22     NO_bypass |-> (!red_op_A && red_op_B && opcode==XOR) | => ##1 (out == $past(^B,2));
23 endproperty
24
25 // Red_op_B assertion - OR
26 property red_op_B_or;
27 @(posedge clk) disable iff(rst)
28     NO_bypass |-> (!red_op_A && red_op_B && opcode==OR) | => ##1 (out == $past(|B,2));
29 endproperty
30
31
32 sequence NO_bypassANDred;
33 !(bypass_A || bypass_B || red_op_A || red_op_B );
34 endsequence
35
36 property OR_prop;
37 @(posedge clk) disable iff(rst)
38     NO_bypassANDred |-> (opcode==OR) | => ##1 (out == ($past(A,2) | $past(B,2)));
39 endproperty
40
41 // Normal XOR (no reduction)
42 property XOR_prop;
43 @(posedge clk) disable iff(rst)
44     NO_bypassANDred |-> (opcode==XOR) | => ##1 (out == ($past(A,2) ^ $past(B,2)));
45 endproperty
46
47 // ADD
48 property ADD_prop;
49 @(posedge clk) disable iff(rst)
50     NO_bypassANDred |-> (opcode==ADD) | => ##1
51     out == ($past(cin,2)? $past(A,2) + $past(B,2)+3'd1 :$past(A,2) + $past(B,2));
52 endproperty
53
54 // MUL
55 property MUL_prop;
56 @(posedge clk) disable iff(rst)
57     NO_bypassANDred |-> (opcode==MULT ) | => ##1
58     (out == ($past(A,2) * $past(B,2)));
59 endproperty
60

```


Questasim Output

1. Waveform



2. Assertion Coverage

Cover Directives											
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Ind	
▲ /top/DUT/ALSU_SVA_inst/#ublk#25906369#19/i_cover__21	SVA	✓	Off	49	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/p_invalid_out_zero_coverage	SVA	✓	Off	938	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/p_valid_leds_zero_coverage	SVA	✓	Off	691	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/bypassA_coverage	SVA	✓	Off	180	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/bypassB_coverage	SVA	✓	Off	140	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/red_op_A_xor_coverage	SVA	✓	Off	22	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/red_op_A_or_coverage	SVA	✓	Off	2	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/red_op_B_xor_coverage	SVA	✓	Off	26	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/red_op_B_or_coverage	SVA	✓	Off	2	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/OR_coverage	SVA	✓	Off	2	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/XOR_coverage	SVA	✓	Off	222	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/ADD_coverage	SVA	✓	Off	74	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/MUL_coverage	SVA	✓	Off	62	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/SHIFT_R_coverage	SVA	✓	Off	40	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/ROTATE_R_coverage	SVA	✓	Off	54	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/SHIFT_L_coverage	SVA	✓	Off	32	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/ROTATE_L_coverage	SVA	✓	Off	20	1	Unli...	1	100%		✓	
▲ /top/DUT/ALSU_SVA_inst/invalid_coverage	SVA	✓	Off	1144	1	Unli...	1	100%		✓	

3. Functional Coverage

INST Valsu_CovCollector_pkg::alsu_covcollector::Co...	100.00%	100	100.00...		
CVP A_CP	100.00%	100	100.00...		
CVP B_CP	100.00%	100	100.00...		
CVP ALU_cp	100.00%	100	100.00...		
CVP cin_CP	100.00%	100	100.00...		
CVP direction_CP	100.00%	100	100.00...		
CVP serial_in_CP	100.00%	100	100.00...		
CVP red_op_A_cp	100.00%	100	100.00...		
CVP red_op_B_cp	100.00%	100	100.00...		
CROSS A_B_ALU_cross	100.00%	100	100.00...		
CROSS CIN_ADD_cross	100.00%	100	100.00...		
CROSS DIRECTION_ALU_cross	100.00%	100	100.00...		
CROSS SERIAL_IN_SHIFT_cross	100.00%	100	100.00...		
CROSS RED_OP_A_OR_XOR_cross	100.00%	100	100.00...		
CROSS RED_OP_B_OR_XOR_cross	100.00%	100	100.00...		
CROSS RED_INVLAID_cross	100.00%	100	100.00...		
/shift_reg_coverage_pkg/shift_reg_coverage	100.00%				
TYPE CovCode	100.00%	100	100.00...		
CVP CovCode::serial_in_CP	100.00%	100	100.00...		
CVP CovCode::direction_CP	100.00%	100	100.00...		
CVP CovCode::mode_cp	100.00%	100	100.00...		

4. ALSU Code Coverage

Statement Coverage:

```
✓ 23 assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
✓ 24 assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
✓ 25 assign invalid = invalid_red_op | invalid_opcode;
✓ 28 always @(posedge clk or posedge rst) begin
✓ 30   cin_reg <= 0;
✓ 31   red_op_B_reg <= 0;
✓ 32   red_op_A_reg <= 0;
✓ 33   bypass_B_reg <= 0;
✓ 34   bypass_A_reg <= 0;
✓ 35   direction_reg <= 0;
✓ 36   serial_in_reg <= 0;
✓ 37   opcode_reg <= 0;
✓ 38   A_reg <= 0;
✓ 39   B_reg <= 0;
✓ 41   cin_reg <= cin;
✓ 42   red_op_B_reg <= red_op_B;
✓ 43   red_op_A_reg <= red_op_A;
✓ 44   bypass_B_reg <= bypass_B;
✓ 45   bypass_A_reg <= bypass_A;
✓ 46   direction_reg <= direction;
✓ 47   serial_in_reg <= serial_in;
✓ 48   opcode_reg <= opcode;
✓ 49   A_reg <= A;
✓ 50   B_reg <= B;
✓ 55 always @(posedge clk or posedge rst) begin
✓ 57   leds <= 0;
✓ 60   leds <= ~leds;
✓ 62   leds <= 0;
✓ 67 always @(posedge clk or posedge rst) begin
✓ 69   out <= 0;
✓ 73   out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
✓ 75   out <= A_reg;
✓ 77   out <= B_reg;
✓ 79   out <= 0;
✓ 84   out <= (INPUT_PRIORITY == "A")? !A_reg: !B_reg;
✓ 86   out <= !A_reg;
✓ 88   out <= !B_reg;
✓
-E 94   out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
✓ 96   out <= ^A_reg;
✓ 98   out <= ^B_reg;
✓ 100  out <= A_reg ^ B_reg;
✓ 102  3'h2: out <= (FULL_ADDER == "ON")? A_reg + B_reg + cin_reg : A_reg + B_reg ; //Fixed Bug: added FULL_ADDER checker
✓ 103  3'h3: out <= A_reg * B_reg;
✓ 109  out <= out_shift_reg;
✓ 116  out <= out_shift_reg;
-E 118 default: out <= 0;
```

Branch Coverage:

ALSU.v

```
29 if(rst) begin
40 end else begin
56 if(rst) begin
58 end else begin
59 if (invalid)
61 else
68 if(rst) begin
71 else begin
72 if (bypass_A_reg && bypass_B_reg)
74 else if (bypass_A_reg)
76 else if (bypass_B_reg)
78 else if (invalid)
80 else begin
82 3'h0: begin
83 if (red_op_A_reg && red_op_B_reg)
85 else if (red_op_A_reg)
87 else if (red_op_B_reg)
89 else
92 3'h1: begin
93 if (red_op_A_reg && red_op_B_reg)
95 else if (red_op_A_reg)
97 else if (red_op_B_reg)
99 else
102 3'h2: out <= (FULL_ADDER == "ON")? A_reg + B_reg + cin_reg : A_reg + B_reg ; //Fixed Bug: added FULL_ADDER checker
103 3'h3: out <= A_reg * B_reg;
104 3'h4: begin
111 3'h5: begin
118 default: out <= 0;
```

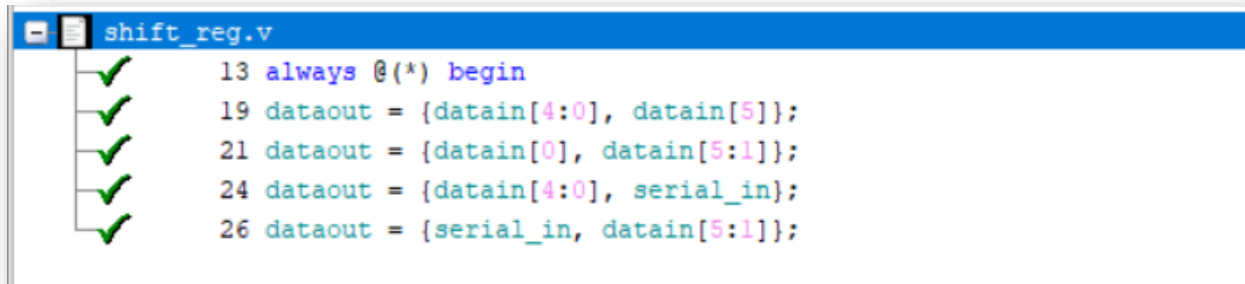
Toggle Coverage:

sim:/top/DUT

- ✓ A
- ✓ A_reg
- ✓ B
- ✓ B_reg
- ✓ bypass_A
- ✓ bypass_A_reg
- ✓ bypass_B
- ✓ bypass_B_reg
- ✓ cin
- ✓ cin_reg
- ✓ clk
- ✓ direction
- ✓ direction_reg
- ✓ invalid
- ✓ invalid_opcode
- ✓ invalid_red_op
- ✓ leds
- ✓ opcode
- ✓ opcode_reg
- ✓ out
- ✓ out_shift_reg
- ✓ red_op_A
- ✓ red_op_A_reg
- ✓ red_op_B
- ✓ red_op_B_reg
- ✓ rst
- ✓ serial_in
- ✓ serial_in_reg

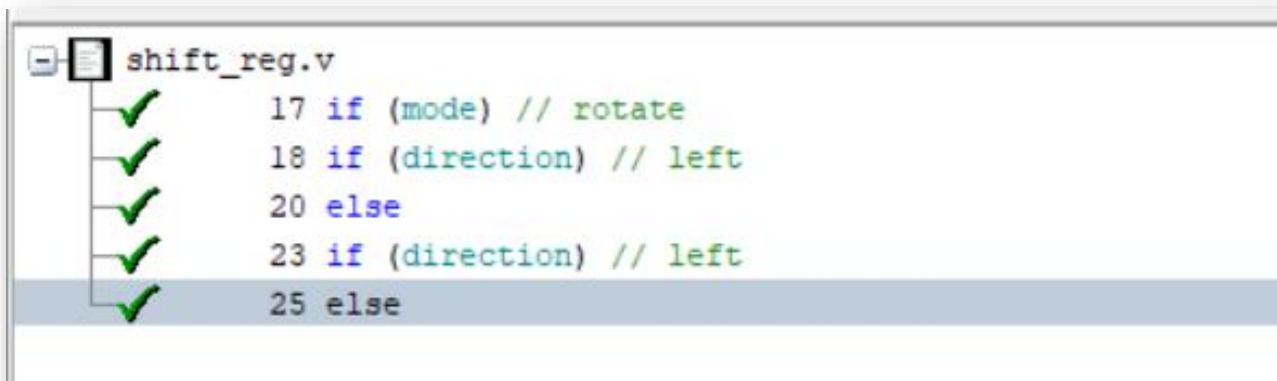
5. Shift Register Code Coverage

Statement Coverage



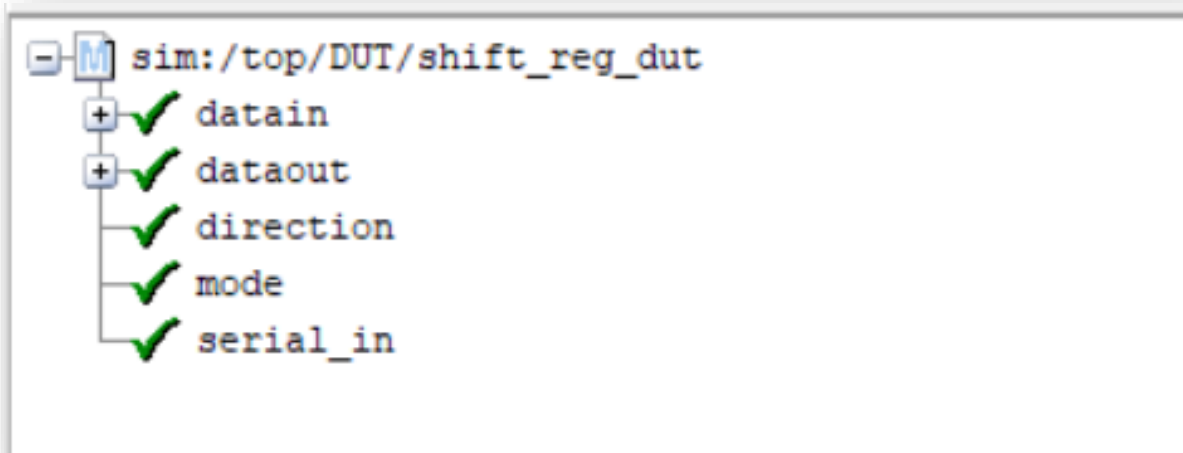
```
shift_reg.v
13 always @(*) begin
19 dataout = {datain[4:0], datain[5]};
21 dataout = {datain[0], datain[5:1]};
24 dataout = {datain[4:0], serial_in};
26 dataout = {serial_in, datain[5:1]};
```

Branch Coverage:



```
shift_reg.v
17 if (mode) // rotate
18 if (direction) // left
20 else
23 if (direction) // left
25 else
```

Toggle Coverage:



```
sim:/top/DUT/shift_reg_dut
+ datain
+ dataout
+ direction
+ mode
+ serial_in
```